생활코딩 머신러닝 실습편

1부: Tensorflow 101 (딥러닝 기초)

with 파이썬 텐서플로

01부

텐서플로 101

KYOBO eBook 12001233963 26.06.29

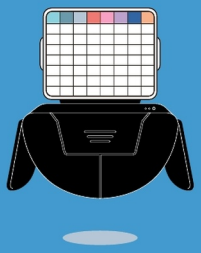
이 수업은 코드로 딥러닝을 구현해보는 딥러닝 기초 수업입니다.
텐서플로를 이용해 가장 간단한 형태의 딥러닝 모델을 작성합니다.
무엇을 넣을까가 아니라, 무엇을 뺄까를 고민하며 만들었습니다.

- **오픈 튜토리얼스 《텐서플로 101》 수업:** https://opentutorials.org/module/4966
- **깃허브 저장소**
  - https://github.com/blackdew/tensorflow1 (1부에서는 이 저장소를 기준으로 설명합니다.)
  - https://github.com/blackdew/ml-tensorflow (1, 2부 코드가 통합된 저장소입니다.)

01장

도입

머신러닝의 주요 용어를 알아보고 머신러닝을 어떻게 공부할지 생각해봅니다. 지도학습의 과정을 간단히 알아보고 실습 환경을 구성합니다.



01 _ 오리엔테이션
02 _ 목표와 전략
03 _ 지도학습의 빅 픽처
04 _ 실습 환경: 구글 코랩

01 https://youtu.be/auCw6qikSYs 06분 03초

오리엔테이션

지금부터 텐서플로(TensorFlow)를 이용해 인공 신경망, 다른 말로는 딥러닝을 구현하는 수업을 시작하겠습니다.

선수 지식

이 수업을 듣는 데 필요한 지식은 다음과 같습니다.

- 파이썬(Python) 언어
- 머신러닝 이론

'머신러닝', '지도학습', '모델', '인과관계'와 같은 말의 의미가 무엇인지 모르신다면 오픈튜토리얼스의 머신러닝 입문 수업인 《머신러닝 1》을 먼저 보신 후 이 수업에 참여하시기를 권합니다.[1]

머신러닝

기계를 학습시켜 인간의 판단 능력을 기계에 위임하는 기술이 **머신러닝**(machine learning)입니다. 머신러닝이라는 이름에는 실로 많은 기술이 포함돼 있습니다. 유명한 것들만 살펴보면 이렇습니다.

1 (엮은이) 《머신러닝 1》 수업은 https://opentutorials.org/module/4916에 있으며, 수업 내용을 책으로 엮은 《생활코딩 머신러닝 이론편》(위키북스, 2021)도 있습니다.

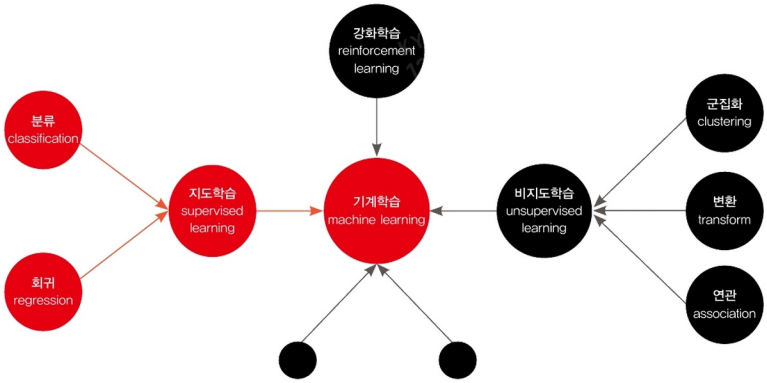


그림 1.1 머신러닝 지도

이 지도는 머신러닝으로 할 수 있는 여러 가지 일들을 보여줍니다. 우리가 텐서플로를 이용해서 해결하려고 하는 문제는 **지도학습**(supervised learning) 영역의 **회귀**(regression) 문제와 **분류**(classification) 문제입니다. 회귀는 숫자로 된 결과를 예측하는 것이고, 분류는 **범주**(category) 형태의 결과를 예측하는 것입니다.



그림 1.2 회귀와 분류 문제

머신러닝 알고리즘

분류와 회귀 문제를 풀기 위해서 다양한 **머신러닝 알고리즘**이 쓰입니다. 유명한 알고리즘(algorithm)으로는 다음과 같은 것들이 있습니다.

- 결정 트리(decision tree)
- 랜덤 포레스트(random forest)
- k-최근접 이웃(KNN: k-nearest neighbors)
- 서포트 벡터 머신(SVM: support vector machine)
- 인공 신경망(artificial neural networks)

인공 신경망

이 책에서 다루는 알고리즘은 **인공 신경망**입니다. 인공 신경망은 사람의 뇌가 작동하는 방법을 모방해서 기계가 학습할 수 있도록 고안된 알고리즘입니다.

우리 두뇌에는 **뉴런**(neuron)이라고 하는 세포들이 촘촘하게 연결돼 있는데요. 뉴런들로 연결된 신경망을 인공적으로 만들었다는 의미에서 **인공 신경망**이라는 이름을 갖게 되었습니다.



그림 1.3 뉴런으로 이뤄진 신경망

딥러닝

지금은 인공 신경망보다 **딥러닝**(deep learning)이라는 이름이 더 유명한 것 같습니다. 인공 신경망을 깊게 쌓아서 만들었다는 표현으로 딥러닝이라는 단어를 쓰기 시작했고, 이 표현이 오늘날 널리 사용됩니다.

딥러닝, 뉴럴 넷, 인공 신경망, 이 이름들은 모두 인간의 신경을 모방한 이론을 가리키는, 같거나 비슷한 말입니다.

인공지능, 머신러닝, 딥러닝의 관계

사람들은 딥러닝이 매우 어려운 문제들을 아주 잘 해결할 수 있다는 사실을 발견하면서부터 딥러닝이라는 용어를 머신러닝을 대신하는 대표적인 용어로 사용하기 시작했고, 머신러닝도 인공지능을 대신하는 대표로써 사용되기 시작했습니다. 하지만 이런 표현들은 엄연히 다른 것입니다.

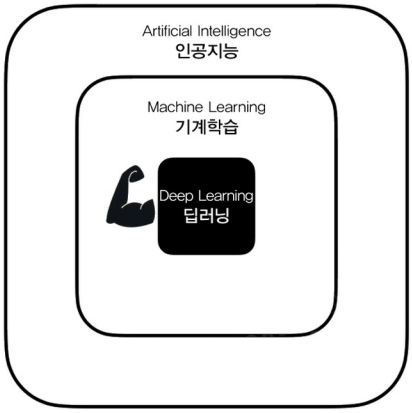


그림 1.4 인공지능, 머신러닝, 딥러닝의 관계

딥러닝 라이브러리

이 '딥러닝'이라는 이론을 공부해서 컴퓨터를 작동시키고 여러 가지 문제를 해결하는 것이 과연 쉬운 일일까요? 아닙니다. 불과 몇 년 전만 해도, 아무리 천재라고 해도 어떤 대기업이라도 할 수 없는 일들이 있었습니다. 그런 어마어마한 일들을 오늘날에는 딥러닝을 이용하여 해낼 수 있게 되었습니다.

그뿐만 아니라 구체적인 딥러닝의 원리를 몰라도 코드만 작성하면 딥러닝으로 문제를 해결할 수 있는 여러 도구가 등장했습니다. 코딩의 세계에서는 이런 도구들을 라이브러리라고 부릅니다.

딥러닝 이론을 프로그래밍 코드로 작성하는 것을 돕는 라이브러리로는 텐서플로, 파이토치, 카페, 테아노 등이 있습니다. 이 라이브러리들은 모두 같은 목적으로 고안된 것으로, 서로 경쟁 관계에 있습니다.

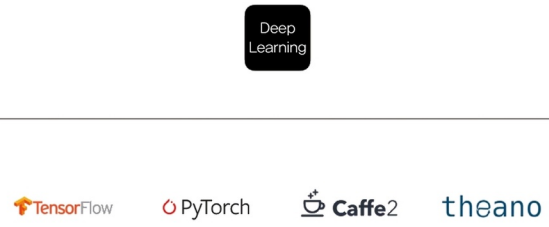


그림 1.5 여러 가지 딥러닝 라이브러리

이 책에서는 텐서플로 라이브러리를 사용해서 딥러닝으로 여러 가지 문제를 해결합니다. 직접 코딩하려면 수만 줄의 코드가 필요한 일을 텐서플로를 이용하면 몇 줄의 코드로 처리할 수 있습니다.



그림 1.6 텐서플로 로고

정리

지금까지의 이야기를 정리해보겠습니다.

- **라이브러리**: 텐서플로의 경쟁자인 파이토치, 카페, 테아노 등은 딥러닝 알고리즘을 코드로 이용할 수 있게 해주는 라이브러리입니다.
- **알고리즘**: 딥러닝, 의사결정 트리, 랜덤 포레스트, KNN, SVM 등의 알고리즘이 있습니다.
- **지도학습**: 우리 수업에서 다루는 회귀와 분류 문제는 지도학습으로 해결할 수 있습니다.
- **머신러닝**: 지도학습, 비지도학습, 강화학습과 같은 기술들을 포괄하는 머신러닝은 오늘날 인공지능을 구현하는 가장 유명하고 유망한 분야라고 할 수 있습니다.

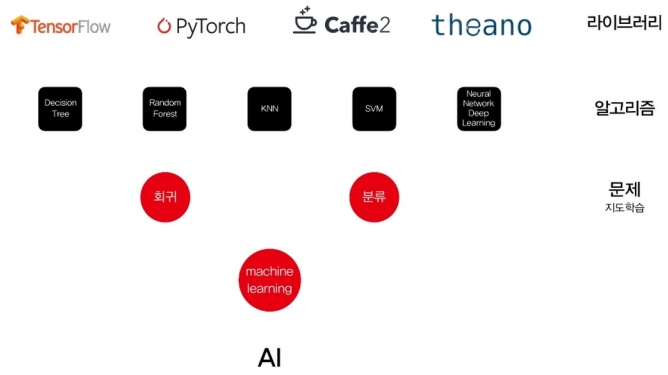


그림 1.7 라이브러리, 알고리즘, 지도학습, 머신러닝

이 개념을 파악하는 것만으로 여러분은 다른 사람과 소통할 수 있고 인공지능에 대한 여러 가지 정보들을 이해할 수 있게 되었습니다. 앞으로 배울 어떤 지식도 여러분을 이렇게 크게 바꿔놓지는 못할 것입니다. 설령 여러분이 이 영상만 보고 남은 수업은 미래로 유보했더라도 사실은 가장 경제적인 공부를 한 것이라는 점을 잊지 마세요.

그럼 지금부터 머신러닝이라는 슈퍼 파워를 구체적으로 배워봅시다.

준비됐나요? 출발하겠습니다.

02 https://youtu.be/IJXQg1btW7w 03분 42초

목표와 전략

이 수업은 딥러닝과 가볍게 만나는 것을 목표로 합니다. 여기서 몇 가지 간단한 딥러닝 코드와 만날 예정인데요, 그 코드들의 사용법을 알고 각자의 데이터로 놀아볼 수 있게 되는 수업을 지향합니다.

딥러닝 입문 강의의 높은 벽

딥러닝을 코드로 구현하는 방법을 제대로 익히는 건 쉬운 일은 아닙니다. 원리, 수학, 코딩, 데이터 등 습득해야 하는 선수 지식이 상당하기 때문입니다.

필자는 현장에서 다음과 같은 순서로 딥러닝 입문 강의를 여러 번 했습니다.

1. 파이썬 코딩의 기초를 배우고,
2. 데이터를 보는 법을 배우고,
3. 머신러닝을 이해하고,
4. 그리고 딥러닝의 원리를 배웁니다.
5. 이렇게 기반 지식을 하나하나 배우고 난 뒤에 드디어 딥러닝 코딩을 배우기 시작합니다.

이런 방법은 충분히 좋고, 우리의 교육 현장에서 많이 사용되고 있는 방법입니다. 그렇지만 이로 인해 딥러닝 입문 수업은 그 이름이 무색하게 난이도가 상당히 높은 과정이 됩니다. 딥러닝에 가볍게 입문해보려는 사람들에게는 높은 벽처럼 느껴질 수밖에 없습니다.

새로운 배움 전략

스마트폰은 매우 복잡한 도구입니다. 그런데 우리가 스마트폰을 사용할 때 스마트폰의 원리나 구현 방법을 공부하지는 않습니다. 이리저리 만져보고, 흔들어보고, 던져보고, 씨름해보

고, 필요한 만큼 경험하면서 비로소 잘 쓸 수 있게 됩니다. 매우 복잡한 도구인 스마트폰을 사용하는 방법을 직접적인 경험을 통해 배워나가는데, 그 과정은 고통스럽지 않고 오히려 즐겁습니다.

딥러닝도 이렇게 자연스럽게 배울 수 있을 것 같았습니다. 이러한 고민 끝에 이 수업에서는 다음과 같은 새로운 전략을 시도하기로 했습니다.

1. 원인이 되는 간단한 코드를 작성하며 경험하고 결과로써 코드의 동작과 학습 과정을 구현합니다. 그리고 해당 코드를 어떻게 이용하면 좋을지 추측해봅니다.
2. 원인이 되는 또 다른 형태의 코드를 작성하고 결과를 구경합니다.

그렇게 몇 가지 코드를 작성하고 결과를 반복하여 구경하면서 딥러닝을 구현하는 시간을 가질 것입니다. 그 과정에서 코드와 알고리즘의 동작에 익숙해지는 것이죠.

혹시 이 전략이 익숙하게 느껴지진 않으신가요? 원인과 결과를 기계에 보여주고 기계가 그 사이의 관계를 스스로 학습하게 하는, 그리고 지금 딥러닝으로 구현해보려 하는 지도학습의 방법과 같습니다.

이 지도학습은 사실 무언가를 배우기 위해 사람들이 아주 익숙하게 사용하는 방법입니다. 그 방법을 기계에 알려준 것에 불과하죠.



그림 1.8 지도학습은 사람이 아주 익숙하게 사용하는 방법입니다

물론 코드로 구현하는 수업이다 보니 원리와 수학, 코딩을 완전히 배제할 수 없다는 한계가 있습니다.

다만 배움을 위해 딱 필요한 만큼만 최대한 경제적으로 알려드릴 생각입니다.

혹시라도 설명이 부족하여 정확히 이해가 되지 않은 부분이 있더라도 수업의 진도를 멈추지는 마세요. 그 정확한 원리의 이해가 바탕이 되지 않더라도 결과에 반복하여 노출되고 익숙해지다 보면 그 원리와 개념을 스스로 깨우쳐 가는 자신을 발견할 수 있을 겁니다.

끝까지 걸어가 봅시다! 여러분을 열심히 응원하겠습니다.

03 지도학습의 빅 픽처

https://youtu.be/dpw0wY13XDk 03분 53초

지금부터 지도학습을 하는 과정을 정리해봅시다. 이 과정이 머릿속에 있다면 어떤 언어나 도구를 사용하더라도 당황하지 않을 것입니다. 이 과정이 머릿속에 없다면 도구가 조금만 달라져도 여러분은 다시 배워야 할 것입니다. 머릿속에 이 과정이 없으면 이해가 잘 되지 않고 쉽게 잊어버리는 것이 당연합니다.

#1 과거의 데이터를 준비합니다

지도학습을 하려면 우선 과거의 데이터가 있어야 합니다.

여러분이 레모네이드 카페를 운영한다고 상상해봅시다. 우리가 장사를 하면서 정리한 데이터는 이렇게 생겼습니다.

표 1.1 온도와 판매량

온도	판매량
20	40
21	42
22	44
23	46

여러분에게는 고민이 있습니다. 레몬을 많이 준비하면 버려야 하고 적게 준비하면 손님을 놓치게 됩니다.

지금 여러분의 꿈은 판매량을 예측하는 것입니다. 가만히 표를 보고 있던 여러분은 온도에 따라서 판매량이 달라진다는 것을 눈치챘습니다. 즉 온도와 판매량 사이에 관계가 있다는 사실을 알아낸 것이죠. 지도학습에서 가장 중요한 순간입니다.

과거의 데이터에서 원인과 결과를 인식하는 데 성공한다면 나머지 복잡한 작업은 머신러닝이 알아서 해줄 겁니다. 데이터 과학과 머신러닝의 분야에서는 이런 원인을 **독립변수**, 결과를 **종속변수**라고 부릅니다.

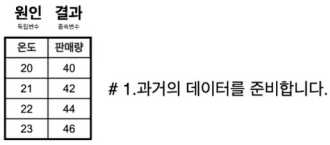


그림 1.9 과거의 데이터를 준비

#2 모델의 구조를 만듭니다

온도를 알려주면 판매량을 예측해주는 기계를 만들어 봅시다. 이런 기계를 머신러닝에서는 **모델**이라고 합니다.

우선 해야 할 일은 모델의 구조를 만드는 것입니다. 독립변수도 하나이고 종속변수도 하나인 데이터를 가지고 있으므로 다음 그림처럼 생긴 모델을 만들면 되겠군요.



그림 1.10 모델의 구조를 만들기

독립변수가 3개, 종속변수가 2개인 모델의 구조를 다음 그림과 같이 나타낼 수 있습니다.

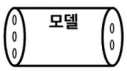


그림 1.11 독립변수가 3개, 종속변수가 2일 때의 모델

#3 데이터로 모델을 학습합니다

데이터를 준비했고 모델의 모양을 만들었으면 이제 모델을 데이터에 딱 맞도록 해야 합니다. 이런 과정을 **핏(fit)**한다고 합니다. 옷이 잘 어울릴 때 핏이 좋다고 하죠. 옷을 입어보는 곳을 피팅 룸이라고도 합니다. 모델을 데이터에 맞게 피팅(fitting)하는 겁니다. 다른 말로는 '학습한다'고 합니다.

이 과정을 통해서 컴퓨터는 온도에 2를 곱하면 판매량이 된다는 사실을 알아냈습니다.

이제 모델이 완성됐네요.

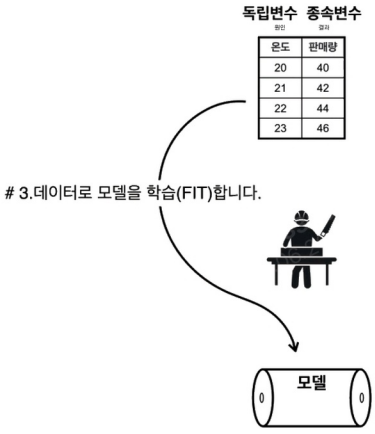


그림 1.12 모델을 학습

#4 모델을 이용합니다

이제 모델을 그냥 이용하면 됩니다.

여러분은 일기예보를 보고 내일의 온도가 15도라는 것을 알아냈습니다. 이것을 모델에게 입력하면 온도 15 × 2를 해서 판매량을 30개로 예측합니다. 그리고 그 결과를 우리에게 알려주죠. 그럼 여러분은 레모네이드 30개 분량의 레몬을 준비하면 되고, 이제 우리는 행복해졌습니다.

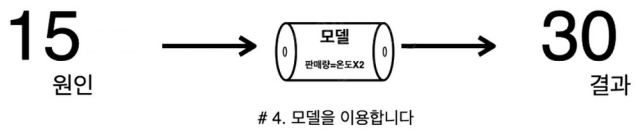


그림 1.13 모델을 이용

정리

이 과정을 다시 떠올려 봅시다.

1. 과거의 데이터를 준비합니다.
2. 모델의 구조를 만듭니다.
3. 데이터로 모델을 학습시킵니다.
4. 모델을 이용합니다.

익숙해질 때까지 계속 반복해서 따라 해보세요. 이것이 확실하다면 우리 수업은 어려울 것이 하나도 없습니다.

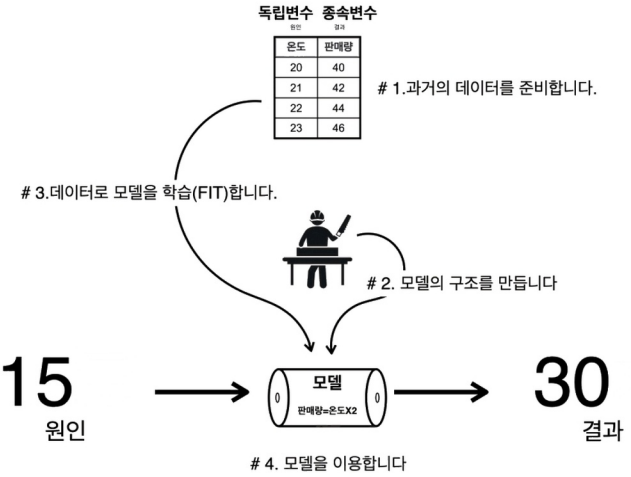


그림 1.14 지도학습의 과정

이제 그림 없이 문장들만 봐도 그 의미가 떠오르는 상태가 되셨나요? 안 떠오른다면 조금만 더 이 자리에 머물러 있자고요.

오늘 하루는 이 과정이 익숙해지는 것만으로도 역사적인 하루가 될 것입니다. 수고하셨습니다.

04 실습 환경: 구글 코랩

https://youtu.be/91E0qenm7W4 05분 12초
실습(구글 드라이브) https://drive.google.com

이 책에서는 구글의 콜라보레이터리(Colaboratory, 이하 '코랩')라는 서비스를 이용합니다. 코랩 환경을 이미 알고 계신다면 바로 2장으로 건너뛰어도 됩니다. 코랩이 뭔지 모르시는 분은 함께 실습할 환경을 준비해보겠습니다.

구글 코랩 소개

데이터 과학과 머신러닝에 사용되는 도구로서 가장 유명한 도구는 주피터 노트북(jupyter notebook)이 있습니다. 노트북이라고 불리는 파일을 웹 브라우저로 실행하여 쉽게 프로그래밍을 해볼 수 있는 환경을 제공하는 도구입니다. 데이터를 다루는 일을 하는 사람들에게 큰 인기가 있습니다. 주피터 노트북을 여러분이 직접 자신의 컴퓨터에 설치할 수도 있고, 온라인 서비스를 통해 이용할 수도 있습니다.

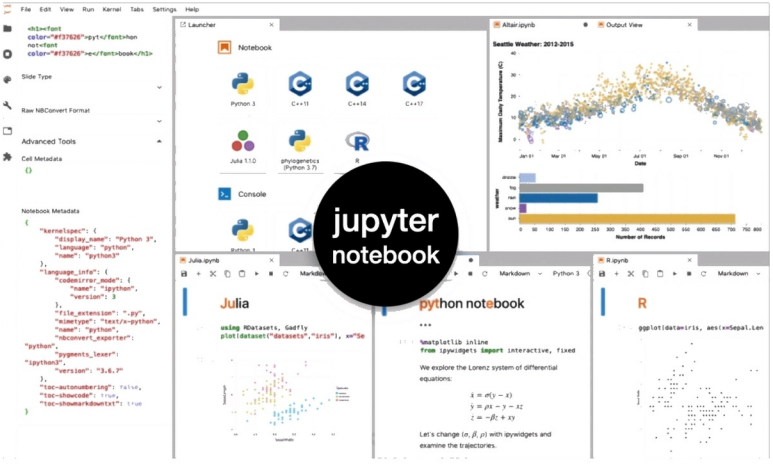


그림 1.15 주피터 노트북

코랩 노트북은 주피터 노트북과 같은 역할을 하는 도구로, 구글 드라이브에서 사용할 수 있도록 구글에서 만든 서비스입니다.

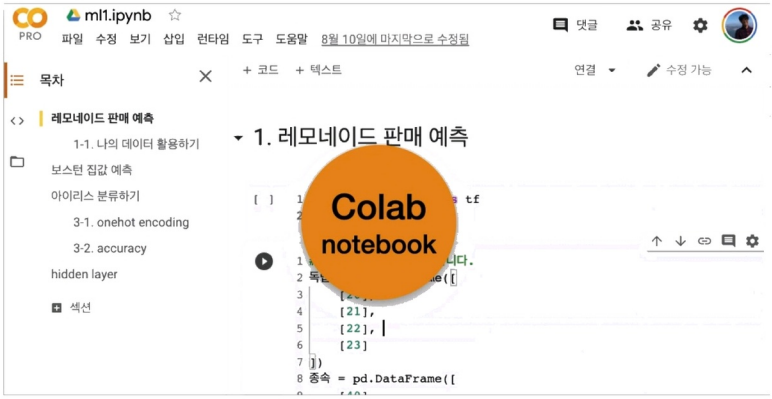


그림 1.16 코랩 노트북

코랩 실습 환경 준비

구글 드라이브(`https://drive.google.com`)에 접속해 **새로 만들기** 버튼을 누른 다음 **더보기 → 연결할 앱 더보기**를 선택합니다.

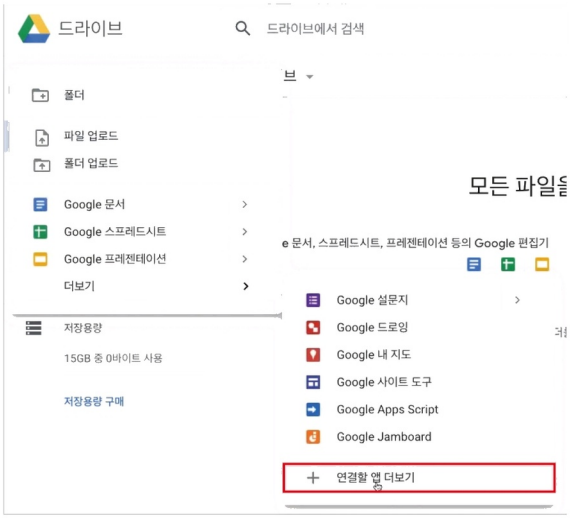


그림 1.17 구글 드라이브의 '연결할 앱 더보기' 메뉴

그러면 앱을 검색할 수 있는 화면이 나타납니다. 여기에 'Colaboratory'라고 입력해 검색하면 검색 결과가 나타납니다.

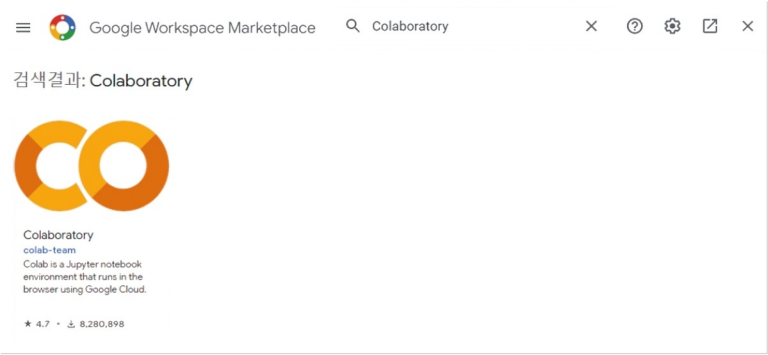


그림 1.18 Google Workspace Marketplace에서 'Colaboratory'를 검색

검색 결과에서 Colaboratory를 클릭하면 코랩을 설치할 수 있는 화면이 열립니다. **설치**를 눌러 코랩 설치를 진행합니다.

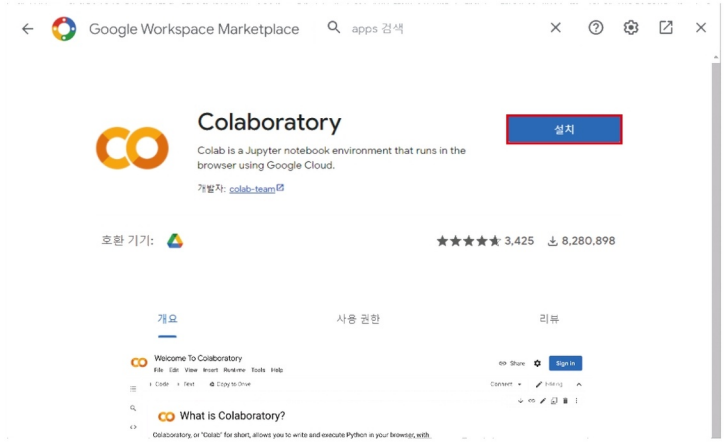


그림 1.19 Colaboratory 설치

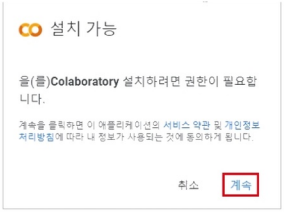


그림 1.20 Colaboratory 서비스 약관 및 개인정보처리방침 동의

설치가 되면 이렇게 설치가 되었다는 표시가 나옵니다.

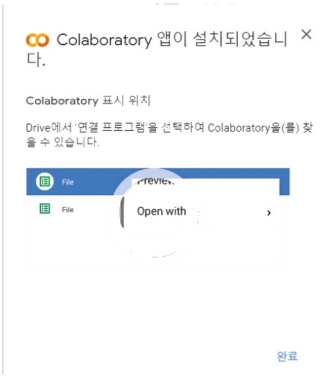


그림 1.21 Colaboratory 설치 완료

코랩 노트북 사용해보기

구글 드라이브에서 **새로 만들기** 버튼을 누르면 **더보기**에 Google Colaboratory 메뉴가 추가되어 있을 것입니다.



그림 1.22 구글 드라이브에서 Colaboratory 노트북 만들기

이 메뉴를 선택하면 구글 드라이브에 코랩 노트북 파일이 하나 생성되고 노트북 환경으로 진입할 수 있습니다.



그림 1.23 코랩 노트북 화면

바로 보이는 커서가 깜박이는 곳을 셀(cell)이라고 합니다. 코드를 작성해 보겠습니다. 코드를 작성한 다음 왼쪽의 실행 버튼(▶)을 누르면 코드가 실행되고 결과가 아래에 나타납니다.

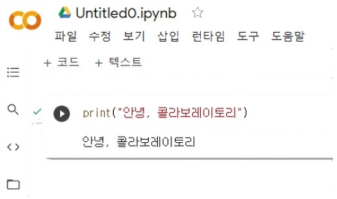


그림 1.24 셀에 코드를 작성하고 실행

셀을 여러 개 추가할 수 있습니다. 마우스 포인터를 셀 위쪽이나 아래쪽 가운데에 갖다 놓으면 코드 셀을 추가하는 버튼이 나타납니다. 또는 상단 메뉴 바로 아래 왼쪽의 + 코드를 클릭해 코드 셀을 추가할 수도 있습니다.



그림 1.25 코드 셀을 추가

코드 셀을 추가해서 코드를 더 작성해보겠습니다.

```
text = 'hello, python'
print(text)
```

실행 결과

```
hello, python
```

실행 버튼을 눌러도 되지만, Ctrl+Enter 혹은 Shift+Enter를 눌러 실행할 수도 있습니다.

- Ctrl+Enter를 누르면 실행한 후에도 셀 안에 머물러 있으므로 같은 셀을 반복해서 실행할 수 있습니다.
- Shift+Enter를 치면 셀을 실행하고 다음 셀로 넘어갑니다. 그래서 순차적으로 셀의 코드를 실행시키면서 진행할 때 사용하면 됩니다.

이렇게 실행이 잘되었으면 실습할 준비가 끝난 것입니다. 노트북 환경이 처음인 분들은 아직 익숙하지 않겠지만, 계속 수업을 이곳에서 할 것이므로 반복해서 이용하다 보면 자연스럽게 익숙해질 것입니다.

소스 코드

실습을 하다 보면 여러 문제가 있을 수 있는데, 그럴 때는 아래 주소에 들어오시면 코드를 비롯해서 수업에 필요한 여러 정보를 찾을 수 있습니다.

- http://opentutorials.org/module/4966

소스 코드가 보이고, 그 위쪽에는 colab과 backend.ai의 링크가 있습니다. colab 링크를 클릭하면 소스 코드를 탑재한 상태로 구글 코랩 서비스가 열리며, 그곳에서 바로 코드를 실행해 실습할 수 있습니다.[2]



그림 1.26 소스 코드 실습 링크(colab과 backend.ai)

준비가 되었으면 첫 번째 코드 실습을 해볼까요?

2 (엮은이) 각 실습이 있는 절의 시작 부분에 해당 코드를 코랩에서 실행할 수 있는 단축 URL을 넣었습니다.

01 https://youtu.be/5qj7ZHc7FDE 04분 22초

판다스

머신러닝의 지도학습 과정 중에서 첫 번째 단계는 과거의 데이터를 준비하는 것이었습니다. 데이터가 있어야 분석을 할 수 있으니까요.

그런데 우리가 가진 데이터는 대부분 파일의 형태로 돼 있습니다. 파일에 들어있는 데이터를 사용하려면 프로그램으로 읽어들이는 과정이 필요합니다. 딥러닝 수업을 시작하기 전에 먼저 이에 해당하는 코드들을 학습하고 실습해보겠습니다.

'변수'의 의미

우선 '변수'라는 용어부터 짚고 넘어가겠습니다.

X = 1이라고 표시하면 이제부터 X는 1입니다.

```
X = 1
```

X = 2라고 표시하면 그때부터 X는 2가 됩니다.

```
X = 2
```

X의 값은 달라질 수 있습니다. 그래서 이런 X를 **변수**(variable)라고 합니다. 프로그래밍을 할 때도 변수라는 용어를 사용합니다.

다음은 프로그램을 작성한 예시인데요. 등호 왼쪽에 있는 것들을 변수라고 합니다. 앞에서 X를 표현한 변수와 모양이 같습니다.

```
# 데이터 불러오기
파일경로 = 'lemonade.csv'
데이터 = pd.read_csv(파일경로)
```

```
파일경로 = 'boston.csv'
데이터 = pd.read_csv(파일경로)

파일경로 = 'iris.csv'
데이터 = pd.read_csv(파일경로)
```

파일경로 변수에 파일의 이름을 담고, 파일을 읽어들인 후, 읽어들인 데이터를 **데이터**라는 변수에 담았습니다. 프로그램의 관점에서 변수는 '데이터를 담고 있다'는 의미가 포함돼 있습니다.

이번에는 표의 관점에서 변수를 확인해보겠습니다. 다음 표에서 온도 값은 20, 21, 22로 달라지는데, 이런 이유로 표에서 칼럼을 '변수'라고 합니다. 이렇게 표에서의 변수는 '관측치가 변한다'는 의미가 있습니다.

표 2.1 온도와 판매량

날짜	요일	온도	판매량
2020.1.3	금	20	40
2020.1.4	토	21	42
2020.1.5	일	22	44

프로그래밍의 관점과 데이터의 관점에서 '변수'라는 용어를 공통으로 사용하지만, 그 사이에는 미묘한 차이가 있다는 것을 이해하셔야 합니다. 대부분 문서에서 변수라는 단어를 혼용하는데, 변수를 데이터의 측면에서 사용한 것인지 프로그램의 관점에서 사용한 것인지 맥락을 알아야 이해할 수 있기 때문입니다.

표를 조금 더 자세히 들여다보면 온도와 판매량은 밀접한 관계가 있는 것이 보입니다. 온도가 20도일 때 40잔이 팔렸고, 21도일 때는 42잔이 팔렸습니다. 원인은 온도, 그리고 결과는 판매량입니다. 이렇게 원인이 되는 변수를 **독립변수**, 결과가 되는 변수를 **종속변수**라고 합니다.

하나의 표 안에는 독립변수와 종속변수가 함께 표현되어 있는데, 지도학습은 이 두 가지를 구분하는 데서 시작합니다. 프로그래밍을 이용하여 지도학습을 할 때도 마찬가지로 독립변수와 종속변수를 분리하는 작업이 필요합니다.

다음은 데이터로부터 독립변수인 온도를 떼어서 **독립**이라는 변수에 담고, 종속변수인 판매량을 떼어서 **종속**이라는 변수에 담는 코드입니다.

```
# 데이터 불러오기
파일경로 = 'lemonade.csv'
데이터 = pd.read_csv(파일경로)

# 독립변수와 종속변수 분리
독립 = 레모네이드[['온도']]
종속 = 레모네이드[['판매량']]

# 데이터 모양 확인
print(독립.shape, 종속.shape)
```

판다스

우리가 이렇게 쉽게 표를 불러와서 분리할 수 있는 것은 표를 다루는 환상적인 판다스(pandas) 라이브러리 덕분입니다.

판다스를 사용하려면 가장 윗부분에 다음과 같은 코드를 추가합니다. 이 코드를 실행한 뒤부터는 지금까지 배웠던 대로 데이터를 불러오고 분리하는 코드를 사용할 수 있게 됩니다.

```
import pandas as pd
```

참고로 판다스는 텐서플로와는 무관한 도구이지만, 실습에서 사용하므로 미리 언급했습니다.

실습 데이터

이 수업에서는 다음 세 가지 데이터를 사용합니다.

- 레모네이드 판매 예측을 위한 데이터: `lemonade.csv`
- 보스턴 집값 예측을 위한 데이터: `boston.csv`
- 붓꽃(iris) 품종 분류를 위한 데이터: `iris.csv`

해당 데이터는 아래 주소의 깃허브에 올려 두었습니다. 이 데이터들을 읽어들여서 독립변수와 종속변수로 분리하는 코드를 먼저 실습해 보겠습니다.

- https://github.com/blackdew/tensorflow1/tree/master/csv
  (단축 주소: https://bit.ly/2PCYMuW 또는 https://bit.ly/tf101csv)

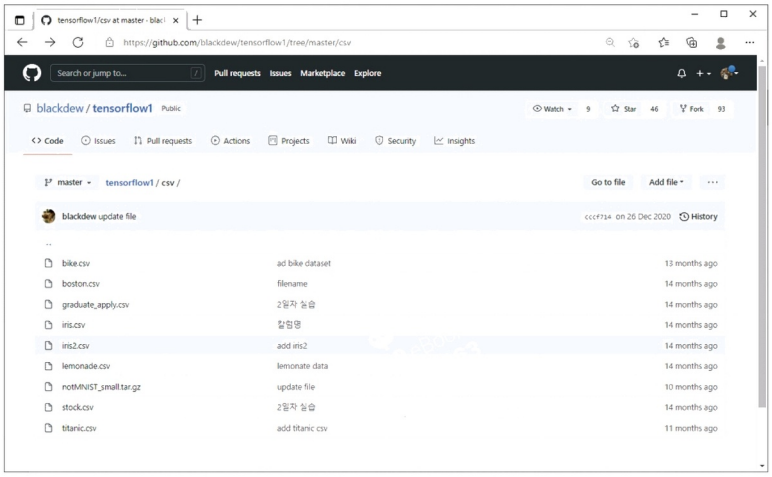


그림 2.1 데이터 파일이 있는 폴더(폴더명: csv)

02 https://youtu.be/spww4gG88Zc 10분 47초
실습(코랩) https://bit.ly/tf101-p1

판다스 실습

판다스 라이브러리로 표를 다루는 실습을 해보겠습니다.

판다스 라이브러리는 표를 다루는 데 유용한 기능들을 제공해주는 라이브러리입니다. 판다스를 사용하면 표의 형태로 돼 있는 데이터를 쉽게 다룰 수 있습니다.

이번 실습에서 배울 것은 다음 5가지입니다.

- **파일 읽어오기**: 우리가 가진 데이터들은 보통 파일 형태로 돼 있으므로 파일에서 직접 데이터를 읽어 들입니다.
- **모양 확인하기**: 읽어들인 데이터의 모양을 확인합니다.
- **칼럼 선택하기**: 데이터에는 종속변수와 독립변수가 같이 들어있습니다. 그래서 학습을 시키기 위해 종속변수와 독립변수를 분리해서 따로 준비합니다.
- **칼럼 이름 출력하기**: 칼럼을 선택하려면 칼럼 이름을 사용하는데, 칼럼 이름을 일일이 입력하거나 칼럼에 무엇이 있는지 다시 파일로 가서 확인하려면 번거롭습니다. 그래서 칼럼들의 이름을 화면에 출력해주는 도구를 사용합니다. 화면에 출력된 칼럼 이름 중 독립변수에 해당하는 칼럼들을 선택해서 붙여넣으면 오타도 줄일 수 있고 효율적으로 작업할 수 있습니다.
- **맨 위 5개 관측치 출력하기**: 데이터가 어떤 모양으로 들어있는지 확인합니다.

실습 코드와 데이터

실습에 사용할 코드는 다음 주소에 있습니다.

- **코드(노트북)**
  깃허브: https://github.com/blackdew/tensorflow1/blob/master/practice1-pandas.ipynb

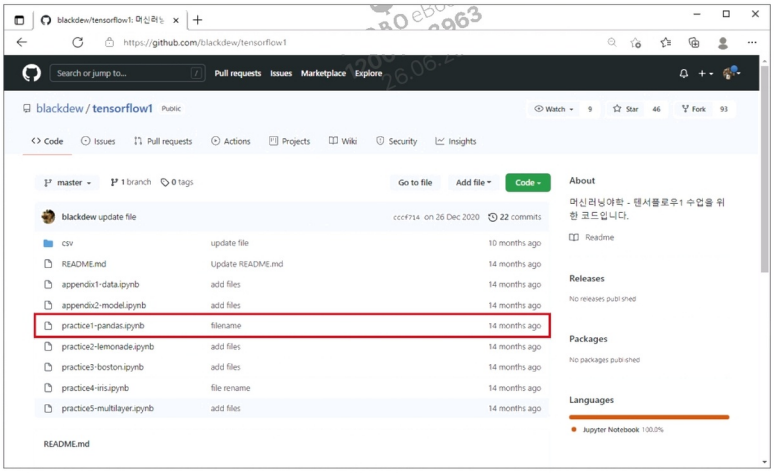


그림 2.2 깃허브 저장소에서 예제 코드 찾기

실습에 사용할 데이터는 다음과 같습니다.

- 레모네이드
  https://raw.githubusercontent.com/blackdew/tensorflow1/master/csv/lemonade.csv
- 보스턴
  https://raw.githubusercontent.com/blackdew/tensorflow1/master/csv/boston.csv
- 아이리스
  https://raw.githubusercontent.com/blackdew/tensorflow1/master/csv/iris.csv

파일 확장자가 csv로 되어 있는데, 이것은 콤마(,)를 이용해서 칼럼들을 구분한 데이터입니다. 엑셀과 같이 표를 보여주는 프로그램에서 csv 파일을 읽어 들이면 테이블 형태로 표시됩니다.

이 파일들은 깃허브에 올려져 있으므로 웹브라우저로 깃허브에 들어가서 각 파일이 어떤 모양인지 직접 확인할 수 있습니다.

데이터 파일 내용 및 URL 확인하는 법

깃허브 사이트에서는 CSV 파일을 다음 그림과 같이 표 형식으로 보여줍니다.

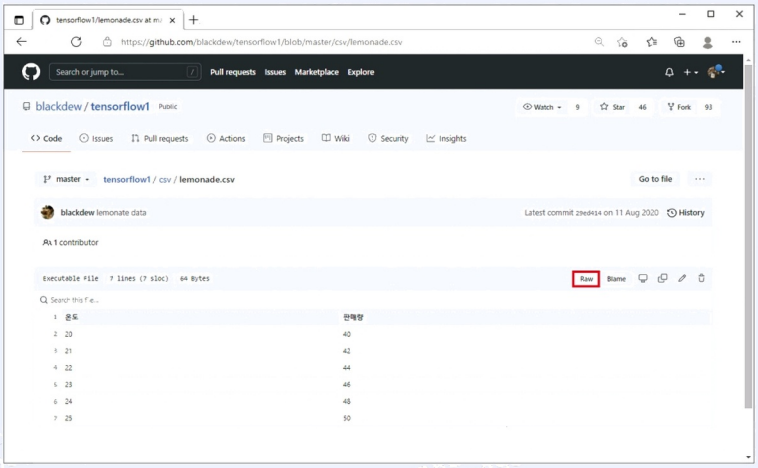


그림 2.3 레모네이드 데이터

표의 오른쪽 위에 있는 Raw 버튼을 클릭하면 CSV 파일의 원래 모습을 볼 수 있습니다. 이때 주소창에 표시된 URL을 복사한 뒤 코드에 붙여넣어 사용할 수 있습니다.

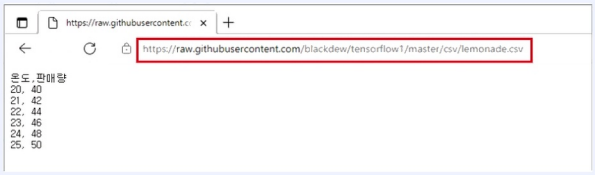


그림 2.4 CSV 파일 내용 및 URL 확인

판다스 라이브러리를 임포트

판다스를 사용하기 위해, 가장 먼저 판다스 라이브러리를 임포트(import)합니다.

```
import pandas as pd
```

파일로부터 데이터 읽어오기

그다음에 파일들로부터 실제로 데이터를 읽어 들여 보겠습니다. **파일경로**라는 변수에다가 각각의 경로를 넣어주고, **pd.read_csv**라는 도구를 통해서 레모네이드 파일을 읽어 옵니다.

```
파일경로 = 'https://raw.githubusercontent.com/blackdew/tensorflow1/master/csv/lemonade.csv'
레모네이드 = pd.read_csv(파일경로)
```

이 두 줄이면 csv로 된 모든 파일을 읽어들여서 이 프로그램에서 사용할 수 있는 형태로 변수에 담아줍니다. 이렇게 실행하면 **레모네이드** 변수에 표 형태의 레모네이드 데이터가 들어옵니다.

다른 파일들도 읽어 들여볼까요?

```
파일경로 = 'https://raw.githubusercontent.com/blackdew/tensorflow1/master/csv/boston.csv'
보스턴 = pd.read_csv(파일경로)
```

파일경로 변수에 보스턴 파일의 경로를 넣고 **read_csv** 함수에 알려주면, 판다스가 그 파일의 데이터를 읽어서 **보스턴** 변수에 담아줍니다.

마찬가지로 아이리스 데이터도 읽어 들입니다.

```
파일경로 = 'https://raw.githubusercontent.com/blackdew/tensorflow1/master/csv/iris.csv'
아이리스 = pd.read_csv(파일경로)
```

그런데 이 파일들이 잘 읽어졌는지 지금은 알 수가 없죠? 변수에 담겨 있을 때는 정말로 그 안에 데이터가 담겨 있는지 눈으로 볼 수가 없습니다.

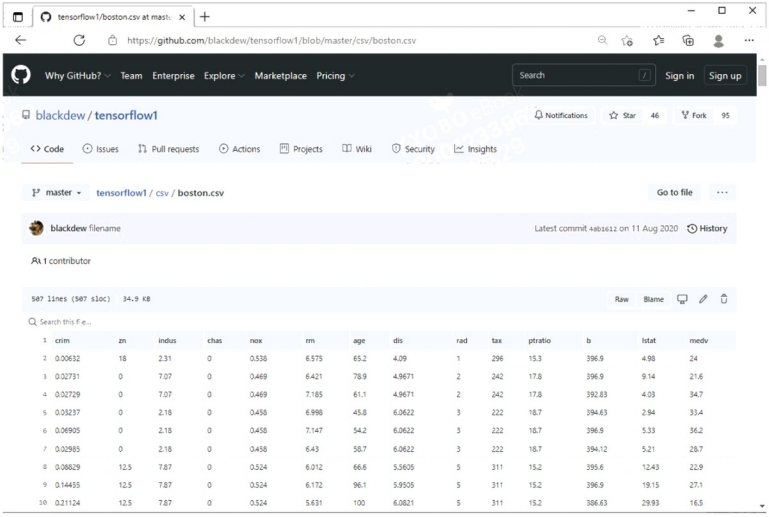
데이터 모양 확인하기

그래서 이 변수를 사용하기 전에 변수에 어떤 데이터가 들어있는지 확인하는 과정이 필요합니다. 가장 쉽게 확인하는 방법은 모양으로 확인하는 겁니다.

판다스에서는 `print()`와 함께 **`레모네이드.shape`**라고 적으면 레모네이드 데이터의 모양이 출력됩니다. 또, **`보스턴.shape`**로 보스턴 데이터의 모양을, **`아이리스.shape`**로 아이리스 데이터의 모양을 확인할 수 있습니다.

코드를 실행하기 전에, 깃허브에 가서 이 모양들이 어떻게 생겼는지 눈으로 확인해볼까요?

아래 보스턴 데이터는 칼럼(column)이 14개, 로우(row)가 507개 있는데, 맨 위 첫 번째 로우는 칼럼명이므로 실제 관측치 데이터는 506개입니다.



	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
2	0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24
3	0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6
4	0.02729	0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
5	0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
6	0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2
7	0.02985	0	2.18	0	0.458	6.43	58.7	6.0622	3	222	18.7	394.12	5.21	28.7
8	0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	395.6	12.43	22.9
9	0.14455	12.5	7.87	0	0.524	6.172	96.1	5.9505	5	311	15.2	396.9	19.15	27.1
10	0.21124	12.5	7.87	0	0.524	5.631	100	6.0821	5	311	15.2	386.63	29.93	16.5

그림 2.5 보스턴 데이터

다른 데이터도 보겠습니다. 레모네이드 데이터가 제일 처음 있었죠. 레모네이드 데이터는 칼럼이 2개, 로우가 6개(7개 중 첫 번째는 칼럼명이므로 제외)로 단순합니다.

Executable File | 7 lines (7 sloc) | 64 Bytes

Search this file...

	온도	판매량
2	20	40
3	21	42
4	22	44
5	23	46
6	24	48
7	25	50

그림 2.6 레모네이드 데이터

다음으로 아이리스는 행이 150개, 칼럼은 5개입니다.

Executable File | 151 lines (151 sloc) | 3.77 KB

Search this file...

	꽃잎길이	꽃잎폭	꽃받침길이	꽃받침폭	품종
2	5.1	3.5	1.4	0.2	setosa
3	4.9	3.0	1.4	0.2	setosa
4	4.7	3.2	1.3	0.2	setosa
5	4.6	3.1	1.5	0.2	setosa
6	5.0	3.6	1.4	0.2	setosa
7	5.4	3.9	1.7	0.4	setosa
8	4.6	3.4	1.4	0.3	setosa
9	5.0	3.4	1.5	0.2	setosa
10	4.4	2.9	1.4	0.2	setosa

그림 2.7 아이리스 데이터

그럼 코드를 실행시켜서 모양들이 제대로 들어왔는지 확인해보겠습니다.

```
print(레모네이드.shape)
print(보스턴.shape)
print(아이리스.shape)
```

실행 결과

```
(6, 2)
(506, 14)
(150, 5)
```

레모네이드의 모양은 (6, 2)로 돼 있고 보스턴의 모양은 (506, 14), 아이리스의 모양은 (150, 5) 이렇게 되어 있네요. 왼쪽이 열이고 오른쪽이 칼럼이니까 방금 보셨던 데이터의 모양과 동일한 것을 알 수 있습니다.

데이터 칼럼 이름 확인

다음으로 독립변수와 종속변수를 분리해야 하는데, 그러려면 칼럼 이름이 필요합니다. 칼럼들의 이름을 출력해보겠습니다.

```
print(레모네이드.columns)
print(보스턴.columns)
print(아이리스.columns)
```

실행 결과

```
Index(['온도', '판매량'], dtype='object')
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax',
       'ptratio', 'b', 'lstat', 'medv'],
      dtype='object')
Index(['꽃잎길이', '꽃잎폭', '꽃받침길이', '꽃받침폭', '품종'], dtype='object')
```

독립변수와 종속변수 분리

그럼 이 이름들을 가지고 종속변수와 독립변수를 분리하겠습니다.

먼저 레모네이드의 독립변수는 **온도** 칼럼 하나이고, 종속변수는 **판매량**이 됩니다. 대괄호를 2개씩 사용한 것에 유의하세요.

```
독립 = 레모네이드[['온도']]
종속 = 레모네이드[['판매량']]
```

이제 데이터가 제대로 분리되었는지 모양을 확인해야 합니다. 출력 결과를 보면 데이터가 6개고, 각각 한 칼럼씩 분리된 것을 볼 수 있습니다.

```
print(독립.shape, 종속.shape)
```

실행 결과

```
(6, 1) (6, 1)
```

이번에는 보스턴 데이터를 분리해보겠습니다. 칼럼들의 의미는 이후 수업에서 설명드리기로 하고, 일단 앞의 13개 칼럼을 독립변수로, 맨 마지막 medv를 종속변수로 하겠습니다. 실행 결과에서 데이터의 개수가 맞는지도 확인해보시기 바랍니다.

```
독립 = 보스턴[['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax',
              'ptratio', 'b', 'lstat']]
종속 = 보스턴[['medv']]
print(독립.shape, 종속.shape)
```

실행 결과

```
(506, 13) (506, 1)
```

마지막으로 아이리스도 분리해보겠습니다. 독립변수로는 앞에 4개의 칼럼을 사용하고, 종속변수로 품종이라는 칼럼 하나를 사용합니다. 이렇게 하면 칼럼별로 분리가 잘되는 걸 볼 수 있습니다.

```
독립 = 아이리스[['꽃잎길이', '꽃잎폭', '꽃받침길이', '꽃받침폭']]
종속 = 아이리스[['품종']]
print(독립.shape, 종속.shape)
```

실행 결과

```
(150, 4) (150, 1)
```

3개의 데이터를 파일로부터 읽어 들여서 독립변수와 종속변수로 분리하는 데이터 준비 과정을 마쳤습니다. 이렇게 분리된 데이터를 가지고 모델을 만들어서, 모델에 이 데이터들을 학습시키면 됩니다.

각각의 데이터 확인해보기

우리가 이후 수업에 사용할 필요한 기능들은 모두 배웠는데, 추가로 한 가지만 더 알아보겠습니다. head()를 사용하면 실제 데이터를 출력할 때 상위 5개의 데이터만 테이블 형태로 볼 수 있습니다.

```
레모네이드.head()
```

실행 결과

	온도	판매량
0	20	40
1	21	42
2	22	44
3	23	46
4	24	48

보스턴 데이터도 마찬가지로 head()로 5개 데이터를 출력해보겠습니다. 보스턴 데이터에는 506개 로우가 있는데 모두 출력해서 보는 건 크게 의미가 없으므로, 어떤 데이터들이 들어있는지 대략적으로 눈으로 확인하기 위해서 head()로 데이터를 출력해보는 것입니다.

```
보스턴.head()
```

실행 결과

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

아이리스 데이터도 마찬가지로 출력해보겠습니다.

```
아이리스.head()
```

실행 결과

	꽃잎길이	꽃잎폭	꽃받침길이	꽃받침폭	품종
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

이제 여러분은 판다스를 이용해서 어떤 데이터를 만나더라도 독립변수와 종속변수를 분리해서 데이터를 준비할 수 있게 되었습니다. 수고하셨습니다.

전체 코드

코드 practice1-pandas.ipynb

```
# 라이브러리 사용
import pandas as pd

# 파일로부터 데이터 읽어오기
파일경로 = 'https://raw.githubusercontent.com/blackdew/tensorflow1/master/csv/lemonade.csv'
레모네이드 = pd.read_csv(파일경로)

파일경로 = 'https://raw.githubusercontent.com/blackdew/tensorflow1/master/csv/boston.csv'
보스턴 = pd.read_csv(파일경로)

파일경로 = 'https://raw.githubusercontent.com/blackdew/tensorflow1/master/csv/iris.csv'
아이리스 = pd.read_csv(파일경로)

# 데이터의 모양 확인
print(레모네이드.shape)
print(보스턴.shape)
print(아이리스.shape)

# 데이터 칼럼명 확인
print(레모네이드.columns)
print(보스턴.columns)
print(아이리스.columns)
```

```
# 독립변수와 종속변수 분리
독립 = 레모네이드[['온도']]
종속 = 레모네이드[['판매량']]
print(독립.shape, 종속.shape)

독립 = 보스턴[['crim', 'zn', 'indus', 'chas', 'nox',
              'rm', 'age', 'dis', 'rad', 'tax',
              'ptratio', 'b', 'lstat']]
종속 = 보스턴[['medv']]
print(독립.shape, 종속.shape)

독립 = 아이리스[['꽃잎길이', '꽃잎폭', '꽃받침길이', '꽃받침폭']]
종속 = 아이리스[['품종']]
print(독립.shape, 종속.shape)

# 각각의 데이터 확인해보기
레모네이드.head()
보스턴.head()
아이리스.head()
```